PONG Report – Game implementation and added features

Balthazar Coquard

25/03/2026

1. Introduction

This project consists of the implementation of the classic arcade game PONG using the Processing programming language and programming environment. The objective of the assignment was not only to recreate the original game mechanics but also to extend it by adding several new features that go beyond purely cosmetic modifications. These features aim to enhance gameplay dynamics, increase difficulty, and demonstrate the application of object-oriented programming principles. Some examples of possible features were given – many of them were chosen to be implemented in this project.

2. Basic Functionality of Pong

The original Pong game published by Atari is a two-player game in which each player controls a paddle positioned on opposite sides of the screen. The objective is to hit a ball back and forth across the field and score points when the opponent fails to return the ball.

The core mechanics of the implemented version include:

- Ball movement: The ball moves continuously across the screen horizontally and vertically.
- Wall collisions: The ball bounces off the top and bottom edges of the screen by reversing its vertical direction.
- Paddle collisions: When the ball hits a paddle, it reverses its horizontal direction.
- Scoring system: A player scores a point when the ball exits the screen on the opponent's side.
- Paddle direction : the player control the horizontal movement of the paddles (no vertical movement)
- Game reset: After each point, the ball is reset to the center of the screen.

These mechanics form the base implementation required by the course.

3. Software Design and Architecture

The program is implemented using object-oriented programming concepts (classes, inheritance, …). A base class called Element is used to represent general objects in the game, storing common properties such as position, size, and movement vectors.

Two main classes extend this base class:

- Paddle class: Represents player-controlled paddles. It manages input handling and movement constraints (bounds of the paddles).

- Ball class: Represents the ball and handles movement, collisions (walls, not paddles), and game state logic (e.g., serving and resetting).

Input handling is separated from object behavior. Keyboard events are processed in the main program (i.e 'keyPressed()', 'keyReleased()') and then passed to the paddles (i.e ball-paddle collision), which update their internal state accordingly. This design improves modularity and reusability.

4. Implemented Features

In addition to the base game, four major features were implemented to extend the gameplay and meet the assignment requirements.

4.1 Variable Bounce Angle, via 'offset'

In the original Pong implementation, the ball always reflects at a fixed angle when hitting a paddle. In this project, the bounce angle depends on the position where the ball hits the paddle[1].

If the ball hits:

- the center of the paddle → it continues almost horizontally
- the top of the paddle → it moves upward
- the bottom of the paddle → it moves downward

This is implemented by calculating an offset between the ball's position and the paddle's center and adjusting the vertical velocity accordingly.

Impact:

- Adds player control over ball direction
- Makes gameplay more strategic and dynamic

4.2 Serve System (Game Start Mechanism)

A serve system was introduced to control when the ball starts moving. At the beginning of the game and after each point, the ball remains stationary in the center of the screen.

The game only resumes when the player presses a specific key (SPACE)[2].

Impact:

- Introduces a clear game state (waiting vs playing)
- Improves user experience and control
- Prevents immediate and uncontrolled restarts.

---

[1] For more information, please see function 'resolvePaddleBounce()'.
[2] For more information, please see function 'keyPressed()' and method 'Ball.startServe()'.

- Enabled the easier creation of the introduction screen.

4.3 Progressive Ball Speed Increase

To increase difficulty over time, the speed of the ball gradually increases during each rally (stored as a global variable). The longer the ball remains in play without scoring, the faster it moves.

This is implemented by slightly increasing the ball's velocity at each frame, while keeping it within a reasonable maximum range[3].

Impact:

- Makes the game progressively more challenging
- Encourages faster reactions
- Prevents overly long and slow rallies
- Also see : 4.3 Progressive Paddle Size Shrinking

4.4 Progressive Paddle Size Shrinking

Similarly to the feature "4.3 Progressive Ball Speed Increase", to dynamically increase the difficulty of the game, a shrinking paddle mechanism was implemented. During a rally (i.e., while the ball remains in play), the size of each paddle gradually decreases over time. This reduction continues until a predefined minimum size is reached. When a point is scored, the paddles are reset to their original size.

Each paddle has a height attribute (h) that determines its vertical size. At each frame, this value is slightly reduced using a multiplicative factor ("PADDLE_SHRINK_STEP = 3.0;"), while keeping the paddles playable via an enforced lower bound, also predefined as constant ("PADDLE_MIN_HEIGHT = 55.0;")[4].

Impact:

- The longer a rally lasts, the harder it becomes to successfully return the ball.
- Players are encouraged to adopt more aggressive strategies to avoid prolonged exchanges.
- It increases tension and engagement, especially in later stages of a rally.

4.5 Two-Dimensional Paddle Movement

In the classic version of Pong, paddles can only move vertically. In this implementation, paddles can move both vertically and horizontally, allowing full two-dimensional control[5].

---

[3] For more information, please see function 'resolvePaddleBounce()' and 'applyRallyDifficulty()'
[4] For more information, please see function 'resolvePaddleBounce()' and 'applyRallyDifficulty()'
[5] For more information, please see method 'Paddle. setBounds()' and functions 'checkPaddleCollisions()', 'isBallOverlappingPaddle()'

To maintain game balance:

- Each paddle is constrained to its respective half of the screen
- Movement is limited within defined boundaries

Impact:

- Adds a new level of gameplay freedom
- Requires better positioning and anticipation
- Significantly changes player interaction with the game

4.6 QWERTY / AZERTY Control Adaptation

To improve usability and accessibility, the game includes a keyboard layout selection feature for the left player. At the beginning of the game, a welcome screen is displayed, allowing the player to choose between a QWERTY and an AZERTY control scheme before the match starts[6].

The layout is selected through the following keys:

- pressing Q chooses the QWERTY layout
- pressing A chooses the AZERTY layout

Once the selection is made, the game switches from the welcome state to the playing state and initializes the match accordingly.

5. Conclusion

My personal objective for this project was to develop my first game using Java (through Processing) while applying sound programming practices. Having previously worked with C++, I was already familiar with many fundamental concepts, and this project provided an excellent opportunity to deepen my understanding of a new language within a practical and engaging context.

In addition to the technical aspect, I aimed to design a game that I would personally enjoy playing. This motivation influenced several design choices, particularly the implementation of the QWERTY/AZERTY keyboard feature. This functionality allows seamless gameplay across different keyboard layouts, which is especially useful in my case, as I regularly switch between French and German keyboards.

Overall, this project enabled me to strengthen my programming skills while creating an interactive and personalized application.

[6] For more information, please see function ‘drawWelcomeScreen()’